```
#include <iostream>
#include <vector>
#include <string>
#include <limits>

using namespace std;

// Base class demonstrating abstraction
class Person {
protected:
    int id;
    string name;

public:
    Person(int personId, const string& personName)
        : id(personId), name(personName) {}

    virtual void displayDetails() const = 0;

    virtual ~Person() = default;

    int getId() const {
        return id;
    }

    string getName() const {
        return name;
    }
```

```
};

// Student class demonstrating inheritance
class Student : public Person {
private:
    string course;
    string contact;

public:
    Student(int studentId, const string& studentName,
            const string& studentCourse,
            const string& studentContact)
        : Person(studentId, studentName),
          course(studentCourse),
          contact(studentContact) {}

    void displayDetails() const override {
        cout << "Student ID: " << id
             << "\nName: " << name
             << "\nCourse: " << course
             << "\nContact: " << contact << '\n';
    }
};

// Librarian class demonstrating inheritance
class Librarian : public Person {
private:
    string employeeCode;
```

```
public:
    Librarian(int librarianId, const string& librarianName,
              const string& code)
        : Person(librarianId, librarianName),
          employeeCode(code) {}

    void displayDetails() const override {
        cout << "Librarian ID: " << id
             << "\nName: " << name
             << "\nEmployee Code: " << employeeCode << '\n';
    }
};

// Book class
class Book {
private:
    int bookId;
    string title;
    string author;
    bool available;

public:
    Book(int id, const string& bookTitle,
         const string& bookAuthor)
        : bookId(id),
          title(bookTitle),
          author(bookAuthor),
```

```
      available(true) {}

int getId() const {
    return bookId;
}

string getTitle() const {
    return title;
}

string getAuthor() const {
    return author;
}

bool isAvailable() const {
    return available;
}

void setAvailability(bool status) {
    available = status;
}

void displayDetails() const {
    cout << "Book ID: " << bookId
         << "\nTitle: " << title
         << "\nAuthor: " << author
         << "\nStatus: "
         << (available ? "Available" : "Issued")
```

```
            << '\n';
    }
};

// Transaction class
class Transaction {
private:
    int transactionId;
    int bookId;
    int studentId;
    string issueDate;
    string dueDate;
    string returnDate;
    bool returned;

public:
    Transaction(int tid, int bid, int sid,
                const string& issue,
                const string& due)
        : transactionId(tid),
          bookId(bid),
          studentId(sid),
          issueDate(issue),
          dueDate(due),
          returnDate("Not returned"),
          returned(false) {}

    int getBookId() const {
```

```
    return bookId;
}

int getStudentId() const {
    return studentId;
}

bool isReturned() const {
    return returned;
}

void markReturned(const string& date) {
    returned = true;
    returnDate = date;
}

void displayDetails() const {
    cout << "Transaction ID: " << transactionId
         << "\nBook ID: " << bookId
         << "\nStudent ID: " << studentId
         << "\nIssue Date: " << issueDate
         << "\nDue Date: " << dueDate
         << "\nReturn Date: " << returnDate
         << "\nStatus: "
         << (returned ? "Returned" : "Issued")
         << '\n';
}
};
```

```
// Library class manages all library operations
class Library {
private:
    vector<Book> books;
    vector<Student> students;
    vector<Transaction> transactions;

    int nextTransactionId = 1;

    Book* findBook(int id) {
        for (auto& book : books) {
            if (book.getId() == id) {
                return &book;
            }
        }
        return nullptr;
    }

    Student* findStudent(int id) {
        for (auto& student : students) {
            if (student.getId() == id) {
                return &student;
            }
        }
        return nullptr;
    }
```

```
public:
    // Add a new book
    void addBook() {
        int id;
        string title, author;

        cout << "Enter book ID: ";
        cin >> id;

        cin.ignore(numeric_limits<streamsize>::max(), '\n');

        if (findBook(id) != nullptr) {
            cout << "A book with this ID already exists.\n";
            return;
        }

        cout << "Enter book title: ";
        getline(cin, title);

        cout << "Enter author name: ";
        getline(cin, author);

        books.emplace_back(id, title, author);

        cout << "Book added successfully.\n";
    }

    // Display all books
```

```
void displayBooks() const {
    if (books.empty()) {
        cout << "No books have been added yet.\n";
        return;
    }

    for (const auto& book : books) {
        book.displayDetails();
        cout << "-------------------------\n";
    }
}

// Search books by ID, title, or author
void searchBook() const {
    int choice;

    cout << "\nSearch by:\n";
    cout << "1. Book ID\n";
    cout << "2. Title\n";
    cout << "3. Author\n";
    cout << "Enter choice: ";

    cin >> choice;

    cin.ignore(numeric_limits<streamsize>::max(), '\n');

    bool found = false;
```

```
if (choice == 1) {
    int id;

    cout << "Enter book ID: ";
    cin >> id;

    for (const auto& book : books) {
        if (book.getId() == id) {
            book.displayDetails();
            found = true;
        }
    }
}
else if (choice == 2 || choice == 3) {
    string term;

    cout << (choice == 2
             ? "Enter title: "
             : "Enter author: ");

    getline(cin, term);

    for (const auto& book : books) {
        string value = (choice == 2)
                       ? book.getTitle()
                       : book.getAuthor();

        if (value.find(term) != string::npos) {
```

```
            book.displayDetails();
            cout << "-------------------------\n";
            found = true;
        }
    }
    else {
        cout << "Invalid search option.\n";
        return;
    }

    if (!found) {
        cout << "No matching book found.\n";
    }
}

// Register a student
void registerStudent() {
    int id;
    string name, course, contact;

    cout << "Enter student ID: ";
    cin >> id;

    cin.ignore(numeric_limits<streamsize>::max(), '\n');

    if (findStudent(id) != nullptr) {
        cout << "A student with this ID already exists.\n";
```

```
        return;
    }

    cout << "Enter student name: ";
    getline(cin, name);

    cout << "Enter course: ";
    getline(cin, course);

    cout << "Enter contact details: ";
    getline(cin, contact);

    students.emplace_back(id, name, course, contact);

    cout << "Student registered successfully.\n";
}

// Display all registered students
void displayStudents() const {
    if (students.empty()) {
        cout << "No students registered yet.\n";
        return;
    }

    for (const auto& student : students) {
        student.displayDetails();
        cout << "-------------------------\n";
    }
```

```
}

// Issue a book to a registered student
void issueBook() {
    int bookId, studentId;
    string issueDate, dueDate;

    cout << "Enter book ID: ";
    cin >> bookId;

    cout << "Enter student ID: ";
    cin >> studentId;

    cin.ignore(numeric_limits<streamsize>::max(), '\n');

    Book* book = findBook(bookId);
    Student* student = findStudent(studentId);

    if (book == nullptr) {
        cout << "Invalid book ID.\n";
        return;
    }

    if (student == nullptr) {
        cout << "Student is not registered.\n";
        return;
    }
```

```
    if (!book->isAvailable()) {
        cout << "Book is already issued.\n";
        return;
    }

    cout << "Enter issue date (DD-MM-YYYY): ";
    getline(cin, issueDate);

    cout << "Enter due date (DD-MM-YYYY): ";
    getline(cin, dueDate);

    book->setAvailability(false);

    transactions.emplace_back(
        nextTransactionId++,
        bookId,
        studentId,
        issueDate,
        dueDate
    );

    cout << "Book issued successfully to "
         << student->getName() << ".\n";
}

// Return a previously issued book
void returnBook() {
    int bookId, studentId;
```

```
string returnDate;

cout << "Enter book ID to return: ";
cin >> bookId;

cout << "Enter student ID: ";
cin >> studentId;

cin.ignore(numeric_limits<streamsize>::max(), '\n');

Book* book = findBook(bookId);

if (book == nullptr) {
    cout << "Invalid book ID.\n";
    return;
}

for (auto& transaction : transactions) {
    if (transaction.getBookId() == bookId &&
        transaction.getStudentId() == studentId &&
        !transaction.isReturned()) {

        cout << "Enter return date (DD-MM-YYYY): ";
        getline(cin, returnDate);

        transaction.markReturned(returnDate);
        book->setAvailability(true);
```

```
            cout << "Book returned successfully.\n";
            cout << "Book is now available.\n";
            return;
        }
    }

    cout << "No active issue record found for this "
         << "book and student.\n";
}

// Display all transactions
void displayTransactions() const {
    if (transactions.empty()) {
        cout << "No transactions recorded yet.\n";
        return;
    }

    for (const auto& transaction : transactions) {
        transaction.displayDetails();
        cout << "--------------------------\n";
    }
}
};

// Main function
int main() {
    Library library;
    int choice;
```

```
do {
    cout << "\n";
    cout << "=========================================\n";
    cout << "       LIBRARY MANAGEMENT SYSTEM\n";
    cout << "=========================================\n";

    cout << "1. Add Book\n";
    cout << "2. Display All Books\n";
    cout << "3. Search Book\n";
    cout << "4. Register Student\n";
    cout << "5. Display Students\n";
    cout << "6. Issue Book\n";
    cout << "7. Return Book\n";
    cout << "8. Display Transactions\n";
    cout << "0. Exit\n";

    cout << "Enter your choice: ";

    if (!(cin >> choice)) {
        cout << "Invalid input. Please enter a number.\n";

        cin.clear();
        cin.ignore(numeric_limits<streamsize>::max(), '\n');

        continue;
    }
```

```
switch (choice) {
    case 1:
        library.addBook();
        break;

    case 2:
        library.displayBooks();
        break;

    case 3:
        library.searchBook();
        break;

    case 4:
        library.registerStudent();
        break;

    case 5:
        library.displayStudents();
        break;

    case 6:
        library.issueBook();
        break;

    case 7:
        library.returnBook();
        break;
```

```
            case 8:
                library.displayTransactions();
                break;

            case 0:
                cout << "Exiting Library Management System. Goodbye!\n";
                break;

            default:
                cout << "Invalid choice. Please try again.\n";
        }

    } while (choice != 0);

    return 0;
}
```